



MCQ

1. Which keyword in Java is used to explicitly generate an exception?

- a) throws
 - b) throw
 - c) final
 - d) error
-

2. The object after the throw keyword must be:

- a) Only Exception
 - b) Only RuntimeException
 - c) Any Throwable subclass object
 - d) Only Error
-

3. What is the correct syntax?

- a) throw new ArithmeticException
 - b) throw Exception("error")
 - c) throw new Exception("msg")
 - d) throws new Exception("msg")
-

4. Which of the following is TRUE about the throw keyword?

- a) Can throw multiple exceptions at once
- b) Can throw only one exception at a time
- c) Cannot throw custom exceptions
- d) Must be written in method signature



CJC

Complete Java Classes

by **Kunal Sir**

5. Custom exceptions must extend:

- a) Object
- b) Throwable
- c) ArithmeticException
- d) IOException

6. Where can the throw keyword appear?

- a) Inside method body
- b) With method signature
- c) Outside class
- d) Only in constructors

7. Which of the following is an UNCHECKED exception?

- a) IOException
- b) FileNotFoundException
- c) ArithmeticException
- d) SQLException

8. What happens after a throw statement executes?

- a) Program continues
 - b) Remaining code inside method is skipped
 - c) Only loop stops
 - d) Exception is suppressed
-



CJC

Complete Java Classes

by Kunal Sir

9. What is the output?

```
throw new NullPointerException("Error");  
System.out.println("Hello");
```

- a) Prints Error then Hello
 - b) Prints Hello
 - c) Compilation error
 - d) Only throws exception, Hello not printed
-

10. throw keyword can throw:

- a) Only pre-defined exceptions
 - b) Only user-defined exceptions
 - c) Both user-defined & predefined exceptions
 - d) None
-

11. What type of exception is thrown using:

```
throw new RuntimeException();
```

- a) Checked
 - b) Unchecked
 - c) Both
 - d) None
-

12. Which must be TRUE for a valid throw?

- a) Object must be Throwable
 - b) Object must be Error only
 - c) Object must be Exception only
 - d) No object required
-



CJC

Complete Java Classes

by **Kunal Sir**

13. throw is used for:

- a) Handling exception
 - b) Declaring exception
 - c) Explicitly throwing exception
 - d) Stopping JVM
-

14. After using throw, execution goes to:

- a) next line
 - b) catch block
 - c) finally always
 - d) compiler
-

15. Which scenario best uses throw?

- a) Database operations
 - b) Input validation failures
 - c) Multithreading
 - d) File reading
-

16. What is thrown?

```
throw new Throwable();
```

- a) Compilation error
 - b) Valid – Throwable allowed
 - c) Valid only inside try
 - d) JVM error
-



CJC

Complete Java Classes

by Kunal Sir

17. What is required BEFORE using throw for a CHECKED exception?

- a) Nothing
 - b) Must handle or declare
 - c) JVM handles automatically
 - d) Must extend RuntimeException
-

18. Which exception is UNCHECKED?

- a) NullPointerException
 - b) IOException
 - c) SQLException
 - d) ClassNotFoundException
-

19. Can you write multiple throw statements in one method?

- a) Yes
 - b) No
 - c) Yes, but only one will execute
 - d) Only inside loops
-

20. The main purpose of throw is:

- a) Propagation
 - b) Explicit exception generation
 - c) Error logging
 - d) Thread stopping
-
-



CJC

Complete Java Classes

by Kunal Sir

21. The throws keyword is used for:

- a) Handling exception
 - b) Declaring exception
 - c) Logging exception
 - d) Creating custom exceptions
-

22. throws is written:

- a) Inside method
 - b) Outside class
 - c) With method signature
 - d) Only in constructors
-

23. Which keyword gives the CALLER a chance to handle an exception?

- a) try
 - b) throw
 - c) finally
 - d) throws
-

24. What is the correct syntax?

- a) void test() throw Exception
 - b) void test() throws Exception
 - c) void test() throw new Exception
 - d) throw test()
-



CJC

Complete Java Classes

by **Kunal Sir**

25. throws can declare how many exceptions?

- a) Only one
 - b) Only two
 - c) Multiple
 - d) None
-

26. Which exception MUST be declared using throws?

- a) ArithmeticException
 - b) NullPointerException
 - c) IOException
 - d) ArrayIndexOutOfBoundsException
-

27. Which is TRUE about throws?

- a) Mandatory only for unchecked exceptions
 - b) Mandatory only for checked exceptions
 - c) Mandatory for all exceptions
 - d) Never mandatory
-

28. What does this mean?

`void test() throws IOException`

- a) test() will always throw exception
 - b) test() may throw IOException
 - c) test() handles IOException
 - d) None
-



CJC

Complete Java Classes

by Kunal Sir

29. If a method declares throws, whose responsibility is it to handle the exception?

- a) Callee
 - b) Caller
 - c) JVM
 - d) Compiler
-

30. Which is not allowed?

- a) throws IOException, SQLException
 - b) throws IOException
 - c) throws new IOException()
 - d) throws Exception
-

31. Can constructors use throws?

- a) Yes
 - b) No
-

32. Which keyword reduces the need for multiple try-catch blocks?

- a) throw
 - b) throws
 - c) catch
 - d) final
-



CJC

Complete Java Classes

by **Kunal Sir**

33. What is required when calling a method containing a CHECKED exception without handling it?

- a) nothing
 - b) declare same exception using throws
 - c) JVM auto-handles
 - d) remove exception
-

34. throws keyword helps in:

- a) Exception propagation
 - b) Exception handling
 - c) Exception suppression
 - d) None
-

35. Which is UNCHECKED?

- a) Exception
 - b) IOException
 - c) RuntimeException
 - d) FileNotFoundException
-

36. If a method uses throws, does it guarantee exception will be thrown?

- a) Yes
 - b) No
-



CJC

Complete Java Classes

by Kunal Sir

37. What happens if a CHECKED exception is neither caught nor declared?

- a) Runtime error
 - b) Compilation error
 - c) JVM ignores
 - d) None
-

38. Which is NOT propagated using throws?

- a) IOException
 - b) SQLException
 - c) RuntimeException
 - d) All can be propagated
-

39. throws is followed by:

- a) Exception object
 - b) Exception class
 - c) Error instance
 - d) Throwable object
-

40. Advantages of throws?

- a) Cleaner code
 - b) Delegates exception handling
 - c) Avoids unnecessary try-catch
 - d) All
-



Practical Questions

Q1. Input Age Validation – Custom Exception

Context: A website needs to restrict access to adult-only content.

Task: Build a console app that takes an integer `age` from the user and validates it.

Requirements

- Define a checked custom exception class `UnderAgeException` extends `Exception` with:
 - Constructors: default and one that accepts a message.
 - A meaningful message (e.g., "Age must be 18 or above.").
- Read `age` from standard input.
- If `age < 18`, **use `throw`** to throw `UnderAgeException`.
- If valid, print "Access granted".

Constraints

- Input must be numeric (handle non-numeric inputs gracefully with a friendly message without crashing).
- Only **one** exception is thrown at a time via `throw`.

I/O Format

- **Input:** Single integer (e.g., 17)
- **Output:** Either the exception message or "Access granted".

Edge Cases

- `age = 18` (boundary).
- Negative ages or extremely large values.
- Empty input / whitespace.

Acceptance Criteria



- Custom exception defined and thrown using `throw`.
- Program skips any statements after `throw` inside the same flow.
- Clear, user-friendly output.

Q2. Bank Withdrawal – ArithmeticException

Context: Simulate a basic ATM withdrawal.

Task: Implement balance checking and raise an exception when funds are insufficient.

Requirements

- **Variables:** `double balance`, `double amount`.
- If `amount > balance`, **throw** `new ArithmeticException("Insufficient Funds")`.
- Otherwise deduct and print the remaining balance.

Constraints

- Amount and balance must be non-negative.
- Decimal values allowed.

I/O Format

- **Input:** Two numbers on separate lines: balance, amount.
- **Output:** Either "Remaining balance: X" or exception message.

Edge Cases

- `amount == balance` (should become zero).
- `amount = 0`.
- Very large amounts, floating rounding.

Acceptance Criteria

- Exception is thrown *explicitly* via `throw`.
- No code after the `throw` in the same block executes.



Q3. Username Check – InvalidUsernameException

Context: Enforce username rules in a signup form.

Task: Reject usernames containing digits.

Requirements

- Create `InvalidUsernameException` extends `Exception` (checked).
- Input: a single `String` username.
- If username contains any digit (0-9), **throw** `InvalidUsernameException("Username must contain only letters.")`.
- On success, print "Username accepted".

Constraints

- Empty or whitespace-only usernames should also be rejected with a **different** message (e.g., "Username cannot be empty"). You can use a simple `if` and `throw`.
- Case-insensitive letters allowed, no spaces inside.

I/O Format

- **Input:** A single line (username)
- **Output:** Acceptance message or exception message.

Edge Cases

- "john123", "", " ", "John_Doe" (decide underscore policy and document it).

Acceptance Criteria

- Uses pattern check (e.g., regex) or character scan.
- Throws **only one** exception per invalid entry.



Q4. Division Program – Zero Denominator

Context: Safe math utility for division.

Task: Divide two integers and prevent division by zero.

Requirements

- Read two ints: numerator, denominator.
- If denominator == 0, **throw** new `ArithmeticException("Cannot divide by zero")`.
- Else print the integer quotient and remainder.

Constraints

- Use integer division.
- Print in the format: "Quotient: q, Remainder: r".

I/O Format

- **Input:** Two integers
- **Output:** Result line or exception message.

Edge Cases

- Negative values.
- Very large or very small numbers (int range).
- 0 / nonZero.

Acceptance Criteria

- Uses `throw` for the zero case.
- No further computation after throwing.



Q5. Voting Eligibility – validateVote(int age)

Context: Election commission validation.

Task: Implement a reusable validator.

Requirements

- Define class `VotingException` extends `Exception` (checked).
- Method: `static void validateVote(int age):`
 - If `age < 18`, **throw** new `VotingException("Not eligible to vote")`.
 - Else print "Eligible to vote".
- In main, read age and call `validateVote`.

Constraints

- `validateVote` must contain the `throw`.
- `main` may either catch it or let the JVM show the stack trace (your choice—state which you implement).

I/O Format

- **Input:** One integer
- **Output:** Either eligibility text or exception message.

Edge Cases

- `age = 17, 18, negative numbers`.

Acceptance Criteria

- Correct use of custom checked exception + `throw`.
- Clear separation of validation logic in a method.



Q6. File Reader – Propagate with throws

Context: A utility method reads a configuration file that may be missing.

Task: Declare the checked exception and propagate it.

Requirements

- Method: `void readFile() throws IOException`
 - Try to open and read a file (`data.txt`).
 - Print the first line if present.
- In main, **do not** use try-catch. Call `readFile()` and **declare** `throws IOException` in main too, so it propagates.

Constraints

- Use `java.io` classes (e.g., `BufferedReader`, `FileReader` or `Files.newBufferedReader`).
- If file does not exist, program terminates with stack trace (intended for this exercise).

I/O Format

- **Input:** None (file-based)
- **Output:** First line of the file or runtime stack trace.

Edge Cases

- Empty file.
- File not found.
- File with Unicode/BOM.

Acceptance Criteria

- `readFile()` signature includes `throws IOException`.
- No try-catch in main (propagation demonstrated).



Q7. Multiple Exceptions – Declaring Many with throws

Context: A network module may hit I/O and DB issues.

Task: Show multiple checked exceptions propagation and handling.

Requirements

- Method: `void connect() throws IOException, SQLException`
 - Simulate two risky steps:
 - Network open (may throw `IOException`).
 - DB handshake (may throw `SQLException`).
 - You can simulate by condition flags, randoms, or mock calls that **actually** throw.
- In main, **handle** both using a single `try` with **multi-catch**:
- `try { connect(); }`
- `catch (IOException | SQLException ex) { System.out.println(ex.getMessage()); }`
- Print a success message if both steps pass.

Constraints

- No nested try-catch inside `connect`—you must **declare** using `throws`.

I/O Format

- **Input:** None or simple config flags.
- **Output:** Either success or the exact exception message.

Edge Cases

- Both error conditions true (only the first thrown is observed).
- Intermittent failures.

Acceptance Criteria

- `connect` declares **two** exceptions in `throws`.
- `main` uses **multi-catch** properly.



Q8. Constructor Using throws – Forcing Caller Handling

Context: Object creation may require a driver class to be present.

Task: Make a constructor that may fail and force caller to handle.

Requirements

- Class: Student
 - Constructor: `Student() throws ClassNotFoundException`
 - Inside, call `Class.forName("com.example.Driver")` (or any non-existent class) to intentionally trigger a checked exception.
- In main, demonstrate **two** approaches:
 1. Propagate by declaring `throws ClassNotFoundException` in main.
 2. Handle via try-catch and print a user-friendly message.

Constraints

- Show both styles clearly (two separate mains or commented sections).

I/O Format

- **Input:** None
- **Output:** Either success creation message or handled error message.

Edge Cases

- Replace with an actual class name to show success path.

Acceptance Criteria

- Constructor signature includes `throws`.
- Caller is **forced** to handle or declare.



Q9. Method Chaining Propagation – $A \rightarrow B \rightarrow C$

Context: Layered architecture (Controller $\rightarrow$ Service $\rightarrow$ DAO).

Task: Show exception propagation across multiple layers.

Requirements

- **Methods:**
 - `void methodC()` throws `IOException` – actually throws a new `IOException("C failed")`.
 - `void methodB()` throws `IOException` – calls `methodC()` but **does not** handle it; just declares throws.
 - `void methodA()` throws `IOException` – calls `methodB()`; also declares throws.
- In `main`, **handle** the exception with try-catch and print:
 - "Handled in main: <message>"
 - Print stack trace or cause chain (your choice).

Constraints

- No handling in A or B—pure propagation using `throws`.

I/O Format

- **Input:** None
- **Output:** Message showing handling in `main`.

Edge Cases

- Enhance by wrapping in B or A to add context (optional): e.g., `throw new IOException("B failed", ex)` and see how cause is preserved.

Acceptance Criteria

- Clear demonstration that checked exceptions bubble up through `throws`.
- `main` is the only handler.



Q10. Network Program – Caller Handles Checked Exception

Context: Sending data over a socket may fail.

Task: Declare in the callee, handle in main.

Requirements

- **Method:** `void sendRequest() throws IOException`
 - Simulate or perform an I/O operation (e.g., open a URL connection or write to a file/socket).
 - On failure, throw `IOException` with a meaningful message.
- **In main:**
 - Use try-catch to handle `IOException`.
 - On success: print "Request sent".
 - On failure: print "Request failed: <message>".

Constraints

- No swallowing exceptions—always include the message.

I/O Format

- **Input:** Optional target string/URL.
- **Output:** Success or failure message.

Edge Cases

- Timeouts (simulate), unreachable host (simulate), bad path.

Acceptance Criteria

- Method declares `throws IOException`.
- `main` handles it cleanly and continues execution after catch.